

Cours Complet dart

Introduction:

Bienvenue dans ce **cours complet sur Dart**, le langage moderne et puissant développé par Google. Dans ce cours, nous allons explorer :

1. **Qu'est-ce que Dart** et ce qui le distingue des autres langages : JavaScript (utilisé pour React Native), Kotlin (pour Android) et Swift (pour iOS).
2. **Les bases de la programmation** : variables, types, opérateurs et structures de contrôle.
3. **La programmation orientée objet (POO)** : classes, objets, héritage, encapsulation, polymorphisme et interfaces.
4. **Les bonnes pratiques de code** : comment améliorer la lisibilité, la clarté et l'organisation de son code, car un code mal structuré peut nuire à la maintenabilité et aux performances d'une application.
5. **Un bonus spécial**, que je te révélerai à la fin de la vidéo pour aller encore plus loin avec Dart et créer des applications puissantes et professionnelles.

Reste attentif jusqu'au bonus ! Ce cours est conçu pour **les débutants**, mais sera également utile aux **développeurs expérimentés** souhaitant renforcer leurs compétences en Dart.

C'est une question que beaucoup de développeurs se posent :

Pourquoi Google a-t-il créé Dart alors qu'ils disposaient déjà d'Android (avec Java et Kotlin) et d'Angular (avec JavaScript et TypeScript) ?

vous savez que Angular seul Avec des outils comme Ionic ou NativeScript : peut être utilisé pour créer des applications **hybrides** qui tournent sur **iOS, Android et le web** à partir du même code Angular.

Android Permet de développer des **applications mobiles natives** avec **Java** (et plus tard **Kotlin**).

Eh bien, il y a une vraie **logique stratégique** derrière ce choix.

Découvrons ensemble **l'histoire et la vision** qui ont conduit à la création de **Dart**.

L'histoire derrière Dart

Au début des années 2010, le web devenait de plus en plus complexe. Les applications comme **Gmail** ou **Google Maps** utilisaient énormément **JavaScript**, mais ce langage commençait à montrer ses limites :

difficile à maintenir dans les très grands projets,

manque de performances pour les applications lourdes,

et absence d'une vraie **structure orientée objet** comme dans Java ou C#.

Face à ces problèmes, les ingénieurs de Google se sont posé une question simple :

« Et si on créait un nouveau langage pour le web — plus rapide, plus structuré et plus facile à maintenir ? »

C'est ainsi qu'en **2011**, deux ingénieurs de haut niveau chez Google, **Lars Bak** et **Kasper Lund**, ont donné naissance à **Dart**.

L'objectif de Google était clair :

Créer un langage **moderne, rapide, sécurisé et facile à apprendre**, capable à long terme de **remplacer JavaScript** sur le web.

Mais l'histoire ne s'arrête pas là.

En **2017**, Google lance **Flutter**, son framework pour créer des applications **mobiles, web et desktop** à partir d'un seul code source.

Et pour ce projet, ils ont naturellement choisi **Dart** comme **langage principal**, car il est rapide, simple, et parfaitement adapté au développement multiplateforme.

Depuis, Dart a connu une **seconde vie** : c'est le cœur de **Flutter**, un écosystème complet qui permet de créer des applications modernes, performantes et élégantes, avec un seul langage.

Tu as maintenant compris ce qu'est Dart à travers cette histoire. Mais comme on dit, **la répétition est la mère de l'apprentissage**, alors reprenons ces notions pour bien **définir et mémoriser ce qu'est Dart**.

Définition de dart :

Dart est un langage de programmation moderne, orienté objet, créé par Google. Il est spécialement conçu pour développer des applications **multi-plateformes** performantes : mobiles, web et desktop, à partir d'un **seul code source**. Dart est le langage principal derrière **Flutter**, le framework UI de Google, permettant de créer des interfaces élégantes et rapides sur toutes les plateformes.

Caractéristiques clés de Dart :

La **compilation AOT** signifie que le code source est **compilé avant l'exécution**, c'est-à-dire **avant que l'application ne soit lancée**. Le code Dart est transformé en **code machine natif** spécifique à la plateforme (iOS, Android, desktop).

Avantages :

- Les erreurs de type et de syntaxe sont détectées à la compilation, ce qui augmente la sécurité et la stabilité.
- Le code est directement exécutable par le système, sans passer par un interpréteur. Les performances sont très élevées, proches du code natif.

Inconvénients :

- La taille de l'application peut être légèrement plus grande car tout est précompilé.

Dart vs React Native : performances et débogage

Dart offre des performances proches du natif grâce à **AOT (Ahead-Of-Time compilation)**.

Contrairement à React Native, qui passe par un **bridge JS → natif**, ce qui entraîne certaines limitations :

- **Complexité du débogage** : les erreurs peuvent apparaître soit dans le code JavaScript, soit dans le code natif. Il faut souvent analyser les deux pour comprendre le problème.
- **Performance légèrement inférieure au natif** : chaque interaction (animations, clics, requêtes réseau) doit passer par le bridge, ce qui ajoute une légère latence.

La compilation JIT signifie que le code source est **compilé au moment de l'exécution**. Dart traduit le code en instructions machine **pendant que l'application tourne**.

avantages :

- Permet le **hot reload** : tu peux modifier le code pendant que l'application est en cours d'exécution et voir instantanément les changements.
- Très utile pendant le **développement** pour itérer rapidement et tester de nouvelles fonctionnalités.

inconvénients:

- Les performances sont légèrement moins rapides que le code AOT, car la compilation se fait en direct.

Éditeurs et environnements pour coder en Dart

Pour programmer en **Dart**, tu as plusieurs options, allant d'éditeurs complets à des environnements en ligne :

1. Visual Studio Code (VS Code)

- C'est un éditeur léger et très populaire pour le développement Dart et Flutter.
- Il offre des extensions officielles : **Dart** et **Flutter**, qui ajoutent :
 - **Coloration syntaxique** et autocomplétion intelligente (IntelliSense)
 - **Débogage intégré**
 - **Snippets** pour accélérer l'écriture de code
 - **Exécution et tests rapides** depuis l'éditeur

2. DartPad

- C'est un environnement en ligne accessible via navigateur : <https://dartpad.dev>
- Pas besoin d'installation sur ton ordinateur
- Idéal pour **tester rapidement des morceaux de code**, expérimenter les fonctions ou apprendre Dart.
- Limité pour les projets complexes ou l'intégration avec Flutter et des packages externes.

Vérifier si Dart est installé

Avant de commencer à coder sur ton ordinateur, il est important de vérifier que **Dart est correctement installé** :

- Ouvre ton terminal et tape :

dart --version

- Si Dart est installé, tu verras la version affichée, par exemple :

Dart SDK version: 3.8.1 (stable)

Si Dart n'est pas encore installé, il faudra le **télécharger et l'installer** avant de continuer. Tu peux suivre la **vidéo dédiée à ce sujet sur ma chaîne YouTube** pour t'aider pas à pas.

Pour **créer un nouveau projet Dart**, on utilise la commande :

```
dart create nom_du_projet(cours_dart)
```

où `nom_du_projet` correspond au nom que tu souhaites donner à ton projet.

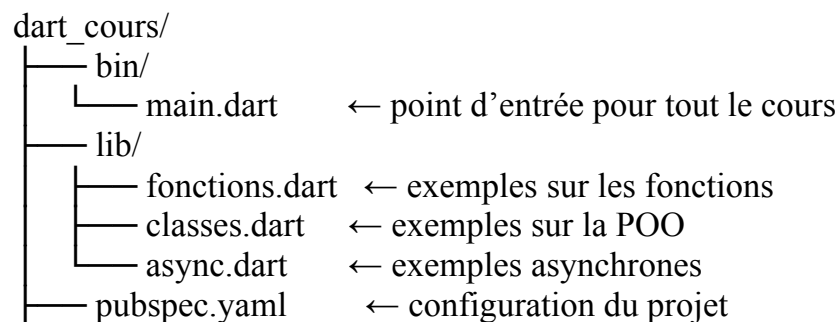
Il est également important **d'activer l'extension Dart dans VS Code** pour bénéficier de toutes les fonctionnalités : coloration syntaxique, autocomplétion, débogage, et exécution directe du code.

Pour ce cours, nous allons utiliser **VS Code** comme éditeur principal pour coder en Dart, car il offre un environnement complet et pratique pour apprendre et expérimenter.

1 Rôle des fichiers dans lib/ et bin/

- **lib/mon_projet.dart** :
 - Contient le **code réutilisable** : classes, fonctions, utilitaires.
 - Tu **n'exécutes jamais ce fichier directement**.
 - Il sert uniquement à être **importé** depuis un fichier dans bin/ ou un autre fichier Dart.
- **bin/mon_projet.dart** :
 - C'est le **fichier exécutable**, le point d'entrée principal.
 - Tu peux faire `dart run bin/mon_projet.dart` pour lancer ton programme.
 - Ce fichier importe le code de lib/.

1 ☐ Structure finale du projet



```
// Pour exécuter : dart run cours_dart_complet.dart ou dart run
```

```
// En Dart, il existe trois types de commentaires :
```

```
// 1   □ Le commentaire sur une seule ligne : commence par `//` et sert à  
expliquer brièvement une instruction.
```

```
// 2   □ Le commentaire sur plusieurs lignes : entouré de `/* ... */`, utile pour  
bloquer ou expliquer un grand bloc de code.
```

```
// 3   □ Le commentaire de documentation : commence par `///`, il est utilisé  
pour documenter les classes, fonctions ou variables afin de générer  
automatiquement une documentation technique.
```

print() sert à afficher du texte dans la console.

\n ajoute une **nouvelle ligne** (retour à la ligne).

Les lignes avec === servent à **rendre l'affichage plus lisible et professionnel**.

```
void main() {  
  print("=== BIENVENUE DANS LE COURS COMPLET DE DART ===\n");
```

```
  // -----
```

```
  // 1   □ VARIABLES ET CONSTANTES
```

```
  // -----
```

```
1   □ Définition d'une variable
```

```
  // -----
```

```
  // Une variable est un espace mémoire qui sert à stocker une valeur.
```

```
  // En Dart, chaque variable a :
```

```
  // - un nom (identifiant)
```

```
  // - un type (int, String, bool, double, etc.)
```

```
  // - et une valeur (le contenu)
```

```
  var nom = "Ciko"; // Le type est déduit automatiquement comme String
```

```
  int age = 25;      // Type explicitement défini
```

```
  double taille = 1.82; // Nombre à virgule
```

```
  bool estDeveloppeur = true; // Booléen (vrai/faux)
```

```
  const pays = "Sénégal";
```

```
  // const : la valeur est définie une fois pour toutes, et connue à la compilation.
```

```
  // Elle ne peut jamais changer, même pendant l'exécution du programme.
```

```
  // Valeur connue et fixée dès la compilation : impossible à modifier.
```

```
  final ville = "Dakar"; // assignée une seule fois (mais à l'exécution)
```

// Le signe \$ permet d'insérer la valeur d'une variable directement dans une chaîne de caractères.

// On appelle cela l'interpolation de chaînes.

// Au lieu d'écrire : print("Nom : " + nom);

// On écrit : print("Nom : \$nom"); plus simple et plus lisible.

```
void main(){
  print("Nom : $nom"); // injecte la valeur de nom
  print("Âge : $age"); // injecte la valeur de age
  print("Taille : $taille m");
  print("Développeur : $estDeveloppeur");
  print("Pays : $pays\n");
}
```

Explication :

\$variable insère la **valeur** de variable directement dans la chaîne (string interpolation).

Si tu veux évaluer une expression ou accéder à une propriété, utilise \${...} :

```
print("Dans 2 ans, tu auras ${age + 2} ans.");
```

```
print("Nom en majuscules : ${nom.toUpperCase()}");
```

```
// -----
// 2  ☐ LIRE UNE SAISIE UTILISATEUR
// -----
```

import 'dart:io'; permet d'utiliser toutes les fonctionnalités d'entrée/sortie (lecture et écriture) dans Dart, comme poser une question à l'utilisateur ou afficher un message dans la console

```
import 'dart:io';
```

```
//   Affiche un message à l'utilisateur sans aller à la ligne (grâce à `stdout.write`)
```

```
// `stdout` = sortie standard (la console)
```

```
// `.write()` = affiche du texte sans saut de ligne automatique
```

```
stdout.write("  Entrez votre prénom : ");
```

```
//   Lit ce que l'utilisateur tape au clavier et l'enregistre dans une variable
```

```
// `stdin.readLineSync()` lit une ligne entrée par l'utilisateur
```

```

// Le point d'interrogation '?' après 'String' signifie que la valeur peut être
`null`
// (par exemple si l'utilisateur appuie juste sur Entrée sans rien écrire)
String? prenom = stdin.readLineSync();

// 💡💡 Affiche un message personnalisé à l'écran avec interpolation de
chaîne
// '$prenom' insère la valeur saisie dans la chaîne de texte

import 'dart:io';

void main() {
// 💡💡 Affiche un message sans retour à la ligne pour demander l'entrée
stdout.write("💡💡 Entrez votre prénom : ");

// 💡💡 Lit ce que l'utilisateur tape au clavier (peut retourner null)
// Le ? permet d'accepter une valeur potentiellement nulle
String? prenom = stdin.readLineSync();

// 💡💡 Affiche un message personnalisé avec interpolation de chaîne
// $prenom insère directement la valeur de la variable dans le texte
print("Salut $prenom, ravi de te rencontrer !\n");
}

// -----
// 4  □ CHAÎNES DE CARACTÈRES (STRING)
// -----

// Déclaration d'une variable de type String (chaîne de caractères)
String phrase = "Hello Dart!";

// 💡💡 .length : renvoie la longueur de la chaîne (le nombre total de
caractères)
print("Longueur : ${phrase.length}"); // Résultat : 11

// 💡💡 .toUpperCase() : transforme toute la chaîne en MAJUSCULES
print("Majuscules : ${phrase.toUpperCase()}"); // Résultat : HELLO DART!

// 💡💡 .toLowerCase() : transforme toute la chaîne en minuscules
print("Minuscules : ${phrase.toLowerCase()}"); // Résultat : hello dart!

// 💡💡 .replaceAll('Dart', 'World') : remplace toutes les occurrences de "Dart"
par "World"

```



```
print("Remplacer : ${phrase.replaceAll('Dart', 'World')}"); // Résultat : Hello
World!
```

```
String phrase = "Hello Dart!";
print("Longueur : ${phrase.length}");
print("Majuscules : ${phrase.toUpperCase()}");
print("Minuscules : ${phrase.toLowerCase()}");
print("Remplacer : ${phrase.replaceAll('Dart', 'World')}\n");
```

```
// -----
```

```
// 5    □ CONVERSION DE TYPES
```

```
// -----
```

```
// ❖❖❖ Déclaration d'une variable de type String contenant des chiffres
String nombreTexte = "123";
```

```
// ❖❖❖ Conversion d'une chaîne de caractères en entier (int)
// La fonction int.parse() transforme une chaîne numérique valide en un entier.
int enInt = int.parse(nombreTexte); // Résultat : 123
```

```
// ❖❖❖ Conversion d'une chaîne de caractères en nombre décimal (double)
// double.parse() convertit une chaîne contenant un nombre avec décimales.
double enDouble = double.parse("3.14"); // Résultat : 3.14
```

```
// ❖❖❖ Affichage des résultats
print("String → int : $enInt");    // Affiche : String → int : 123
print("String → double : $enDouble"); // Affiche : String → double : 3.14
```

```
// ❖❖❖ Conversion inverse : int → String
// La méthode toString() convertit n'importe quel type en texte (String)
print("int → String : ${enInt.toString()}"); // Affiche : int → String : 123
```

```
String nombreTexte = "123";
int enInt = int.parse(nombreTexte);
double enDouble = double.parse("3.14");
```

```
print("String → int : $enInt");
print("String → double : $enDouble");
print("int → String : ${enInt.toString()}\n");
```

```
// -----
```

```
// 6    □ OPÉRATEURS ARITHMÉTIQUES
```

```
// -----
```

Je remarque que certains développeurs, lorsqu'il s'agit de réaliser des opérations arithmétiques, écrivent souvent :

```
int a = 10, b = 3;
print("a + b = ${a + b}");
```

Oui, le code fonctionne et affiche correctement le résultat, mais **ce n'est pas une bonne pratique**.

En effet, les noms a et b ne donnent **aucune indication sur la signification des valeurs** : s'agit-il de prix, de notes, de quantités... ? On ne sait pas.

En programmation, il est essentiel de respecter deux notions fondamentales :

1 ☐ **Lisibilité du code** : le code doit être clair, compréhensible et facile à suivre, **pour toi comme pour les autres développeurs**.

2 ☐ **Clarté des noms** : les variables doivent décrire **leur rôle ou leur contenu**, sans avoir besoin d'un commentaire pour comprendre ce qu'elles représentent.

```
int a = 10, b = 3;
print("a + b = ${a + b}");
print("a - b = ${a - b}");
print("a * b = ${a * b}");
print("a / b = ${a / b}");
print("a ~/ b = ${a ~/ b} (division entière)");
print("a % b = ${a % b} (reste)\n");
```

La **bonne pratique** consiste à utiliser des **noms de variables explicites** qui indiquent clairement leur rôle ou leur signification.

Par exemple, au lieu de noms génériques comme a et b, on peut écrire

```
void main() {
La bonne pratique sera de faire
//   Noms explicites pour les variables
int nombrePrincipal = 10;
int nombreSecondaire = 3;

//   Opérations arithmétiques de base
print("Addition : $nombrePrincipal + $nombreSecondaire =
${nombrePrincipal + nombreSecondaire}");
print("Soustraction : $nombrePrincipal - $nombreSecondaire =
${nombrePrincipal - nombreSecondaire}");
print("Multiplication : $nombrePrincipal * $nombreSecondaire =
${nombrePrincipal * nombreSecondaire}");
print("Division : $nombrePrincipal / $nombreSecondaire =
${nombrePrincipal / nombreSecondaire}");
```

```

print("Division entière : $nombrePrincipal ~/ $nombreSecondaire =
${nombrePrincipal ~/ nombreSecondaire} (partie entière)");
print("Modulo : $nombrePrincipal % $nombreSecondaire =
${nombrePrincipal % nombreSecondaire} (reste)\n");
}

```

Avantages de cette approche :

- Les noms de variables expliquent **immédiatement leur rôle**.
- Le code devient **plus lisible et compréhensible** pour toi et pour d'autres développeurs.
- Cela réduit le besoin de commentaires supplémentaires pour expliquer ce que fait le code.

```

=====
// 3. Structures de contrôle

```

```

// =====
//

```

// **Les structures de contrôle**

Les **structures de contrôle** sont des instructions qui permettent de **modifier le flux d'exécution** d'un programme.

Autrement dit, elles permettent de décider **quelle partie du code sera exécutée, combien de fois et dans quel ordre**.

//   Déclaration d'une variable booléenne

// Elle indique si l'utilisateur a le profil "développeur" ou non

```
bool estDeveloppeur = true;
```

//   Structure conditionnelle : on adapte le contenu affiché selon la valeur de la variable

```
if (estDeveloppeur) {
```

```
    // Si l'utilisateur est développeur, on affiche tout le contenu
```

```
    print("  Affichage du contenu complet pour développeur.");
```

```
} else {
```

```
    // Si l'utilisateur n'est pas développeur, le contenu est restreint
```

```
    print("  Affichage du contenu restreint.");
```

```
}
```

//   Condition utilisant directement une variable numérique

// Ici, on vérifie si l'âge de l'utilisateur est supérieur à 18

```
if (age > 18) {
```

```
    // Si l'âge est supérieur à 18, l'utilisateur a un accès complet
```

```
    print("Accès complet autorisé.");
```

```
} else {
```

```
// Sinon, l'accès est limité
print("Accès limité.");
}
```

```
// =====
// 4. Collections : Listes, Sets et Maps
// =====
```

Une **collection** est une structure de données qui permet de **regrouper plusieurs valeurs ou objets** sous une même entité. Elle sert à **stocker, organiser et manipuler un ensemble d'éléments** dans un programme.

En Dart, les principales collections sont :

1. **List** : une liste ordonnée d'éléments, qui peut contenir des doublons et dont on peut accéder aux éléments par leur **indice**.
2. **Set** : un ensemble d'éléments **uniques**, non ordonné, idéal pour éviter les doublons.
3. **Map** : une collection **clé-valeur**, où chaque valeur est associée à une clé unique, comme un dictionnaire.

```
//   Liste d'entiers : une collection ordonnée qui peut contenir des doublons
List<int> nombres = [1, 2, 3, 4, 5];
```

```
// Accéder à un élément par son indice (les indices commencent à 0)
print(nombres[0]); // Affiche le premier élément : 1
```

```
// Ajouter un élément à la fin de la liste
nombres.add(6);
```

```
// Supprimer un élément spécifique de la liste
nombres.remove(3);
```

```
// Afficher la liste après modifications
print("Liste de nombres : $nombres"); // Affiche : [1, 2, 4, 5, 6]
```

```
// Trouver l'indice d'un élément dans la liste
int index = nombres.indexOf(4);
print("L'indice de 4 est : $index"); // Affiche : 2
```

```
//   Ensemble (Set) : collection non ordonnée, sans doublons
Set<String> pays = {"USA", "Canada", "France"};
```

```

// Ajouter un élément
pays.add("Japon");

// Supprimer un élément
pays.remove("Canada");

// Afficher le contenu du Set
print("Ensemble de pays : $pays"); // Affiche : {USA, France, Japon}

//   Map (dictionnaire) : collection clé-valeur, Collection clé-valeur, très
utile pour représenter des relations (ex : pays → capitale).
Map<String, String> capitales = {
  "France": "Paris",
  "Allemagne": "Berlin",
  "Espagne": "Madrid"
};

// Ajouter une nouvelle clé-valeur
capitales["Italie"] = "Rome";

// Supprimer une clé et sa valeur associée
capitales.remove("Allemagne");

// Afficher le contenu de la Map
print("Capitales : $capitales"); // Affiche : {France: Paris, Espagne: Madrid,
Italie: Rome}

// Accéder à la valeur associée à une clé spécifique
print("La capitale de l'Italie est : ${capitales["Italie"]}");

// =====
//   CHAPITRE 4 : LES FONCTIONS EN DART
// =====
// Une fonction est un bloc de code réutilisable qui exécute une tâche précise.
// Elle peut recevoir des paramètres et retourner une valeur.
// =====

// -----
// 1   ☐ Fonction simple sans retour
// -----

```

Je crée une **fonction simple sans valeur de retour** dans un fichier situé dans le dossier lib :

```
void afficherMessage() {  
  print("💎💎 Bienvenue dans le monde des fonctions en Dart !");  
}
```

Ensuite, je peux **appeler cette fonction depuis un fichier situé dans le dossier bin** en important le package correspondant :

```
// bin/main.dart  
import 'package:mon_projet/fonctions.dart';
```

```
void exempleFonctionSimple() {  
  afficherMessage();  
}
```

Explications / Bonnes pratiques :

1. Placer les fonctions **réutilisables dans lib** permet de mieux organiser le code et de les partager facilement entre différents fichiers.
2. Les fichiers dans bin servent généralement de **point d'entrée du programme**.
3. L'import avec package:mon_projet/fonctions.dart permet d'utiliser la fonction définie dans le package sans recopier le code.

Avantages de cette organisation :

1. **Lisibilité et clarté** : toutes les fonctions réutilisables sont regroupées.
2. **Séparation des responsabilités** : lib contient le code métier, bin le point d'entrée.
3. **Réutilisabilité** : tu peux utiliser ces fonctions dans d'autres fichiers ou projets sans les copier.
4. **Facilité d'évolution** : ajouter de nouvelles fonctions ne casse pas le code existant.

```
// -----  
// 2   □ Fonction avec paramètres  
// -----  
// On peut passer des valeurs à une fonction sous forme de paramètres.  
void afficherNombre(int nombre) {  
  print("💎💎 Mon nombre est : $nombre");  
}  
  
void exempleFonctionAvecParametre() {  
  afficherNombre(42);  
  afficherNombre(100);  
}
```

```
// -----
// 3  □ Fonction qui retourne une valeur
// -----
// Une fonction peut renvoyer un résultat à l'aide du mot-clé `return`.
String creerTexteNombre(int nombre) {
    return "💎💎 Mon nombre est $nombre";
}

void exempleFonctionAvecRetour() {
    String texte = creerTexteNombre(25);
    print(texte);

    // Ou directement
    print(creerTexteNombre(50));
}

// -----
// 4  □ Fonction avec plusieurs paramètres
// -----

int additionner(int a, int b) {
    return a + b;
}

void exempleAddition() {
    int resultat = additionner(10, 20);
    print("💎💎 La somme de 10 et 20 est : $resultat");
}
```

En programmation, on distingue deux notions :

- **Lisibilité** du code : le code doit être clair pour toi **et pour les autres développeurs**.
- **Clarté des noms** : les variables doivent **expliquer leur rôle** sans qu'on ait besoin d'un commentaire.

Dans ton exemple :

```
int additionner(int a, int b) {
    return a + b;
}
```

💎💎 Ici, a et b ne disent **rien sur leur signification**.

Est-ce qu'il s'agit de prix ? d'âges ? de notes ? d'heures ?

On ne sait pas.

Bonne pratique :

Utilise des **noms explicites**, même si le code devient un peu plus long.

Exemples :

❖❖ Exemple 1 : Somme de deux nombres

```
int additionner(int premierNombre, int deuxiemeNombre) {  
    return premierNombre + deuxiemeNombre;  
}
```

```
void exempleAddition() {  
    int resultat = additionner(10, 20);  
    print("❖❖ La somme de 10 et 20 est : $resultat");  
}
```

Ici, on comprend tout de suite que la fonction additionne deux nombres.

❖❖ Exemple 2 : Somme de deux montants

```
double additionnerMontants(double montant1, double montant2) {  
    return montant1 + montant2;  
}
```

```
void exempleMontant() {  
    double total = additionnerMontants(1500.50, 2499.99);  
    print("❖❖ Le total des montants est : $total FCFA");  
}
```

Les noms de variables indiquent **le contexte financier**.

```
// -----  
// ❖❖ Fonction principale pour exécuter les exemples  
// -----  
void main() async {  
    print("❖❖ Démonstration des fonctions en Dart\n");  
  
    exempleFonctionSimple();  
    print("-----");  
  
    exempleFonctionAvecParametre();  
    print("-----");  
  
    exempleFonctionAvecRetour();  
    print("-----");  
}
```



```

exempleAddition();
print("-----");

exempleParametresNommes();
print("-----");

exempleParametresOptionnels();
print("-----");

exempleFonctionFlechee();
print("-----");

exempleFonctionAnonyme();
print("-----");

exempleFonctionCommeParametre();
print("-----");

await exempleFonctionAsynchrone();

print("\n❖❖ Fin du cours sur les fonctions !");
}

```

```

// =====
// ❖❖ CHAPITRE 5 : La Programmation Orientée Objet (POO)
// =====
// Dans ce chapitre, nous allons découvrir les principes de base de la POO en
// Dart :
// - Classes et objets
// - Constructeurs
// - Encapsulation (getters / setters)
// - Héritage
// - Polymorphisme
// - Classes abstraites
// - Interfaces
// =====

// -----
// 1  □ Classes et objets : la base
// -----
// Une classe représente un modèle (ou plan) pour créer des objets.
// Un objet est une instance concrète de cette classe.

```

Dans le dossier lib/, on crée un sous-dossier classes/ contenant les fichiers
 personne.dart , compte_bancaire.dart, paiement , véhicule, animal ,chien ,chat .

Pour utiliser ces classes dans notre programme principal, il suffit de les **importer** dans le fichier bin/main.dart.

```
=====
// Classe représentant une personne
// ===
class Personne {
  String nom; // Nom de la personne
  int age;    // Âge de la personne

  // ❖❖ Constructeur
  // Permet de créer un objet Personne avec des valeurs initiales pour le nom et
  // l'âge
  Personne(this.nom, this.age);

  // ❖❖ Méthode
  // Comportement d'un objet Personne : se présenter avec son nom et son âge
  void sePresenter() {
    print("❖❖ Bonjour, je m'appelle $nom et j'ai $age ans.");
  }
}
```

Classe : définit le modèle ou le plan pour créer des objets (ici, une personne).

Attributs : caractéristiques de l'objet (nom, age).

Constructeur : initialise les attributs lorsque l'objet est créé.

Méthode : définit un comportement que l'objet peut exécuter (sePresenter).

```
void exempleClasseEtObjet() {
  // ❖❖ Création d'objets de la classe Personne
  Personne personne1 = Personne("Chrislain", 27);
  // Appel de la méthode sePresenter() pour afficher les informations de l'objet
  personne1.sePresenter();

  Personne personne2 = Personne("Amina", 22);
  // Appel de la méthode sePresenter() pour afficher les informations de l'objet
  personne2.sePresenter();
}
```

Personne personne1 = Personne("Chrislain", 27); crée un **objet** personne1 avec le nom "Chrislain" et l'âge 27.

personne1.sePresenter(); appelle la **méthode** de l'objet pour exécuter son comportement (ici, afficher un message).

Même logique pour personne2.

```
// -----
// 2  □ Encapsulation : cacher les détails internes
// -----
```

Encapsulation en Dart :

C'est le principe de **protéger les données d'une classe** en les rendant **privées** (en préfixant les noms d'attributs par **_**) et de **contrôler leur accès** via des **getters et setters**. Cela permet de garantir que les données ne soient modifiées qu'avec des règles définies, améliorant la sécurité et la maintenance du code.

// On protège les données sensibles en les rendant privées (_variable)
// et on y accède uniquement via des getters / setters.

=====

Classe représentant un compte bancaire

// =====

```
class CompteBancaire {
  // ❖❖ Propriété privée
  // L'attribut _solde est privé : accessible uniquement à l'intérieur de la classe
  double _solde = 0;

  // ❖❖ Getter : permet de lire la valeur de _solde depuis l'extérieur
  double get solde => _solde;

  // ❖❖ Setter : permet de modifier la valeur de _solde avec une validation
  set solde(double montant) {
    if (montant >= 0) {
      _solde = montant; // Mise à jour uniquement si le montant est positif
    } else {
      print(" Le montant ne peut pas être négatif !");
    }
  }

  // ❖❖ Méthode : affiche le solde actuel du compte
  void afficherSolde() {
    print("❖❖ Solde actuel : $_solde FCFA");
  }
}

// ❖❖ Exemple d'utilisation de l'encapsulation
void exempleEncapsulation() {
  CompteBancaire compte = CompteBancaire();
```

```
// Utilisation du setter pour attribuer un solde valide
compte.solde = 10000;
compte.afficherSolde(); // Affiche : 10000 FCFA

// Tentative d'attribuer un solde négatif (déclenche validation)
compte.solde = -500; // Affiche le message d'erreur
}
```

Encapsulation : protéger les données d'un objet en les rendant privées (`_solde`) et contrôler l'accès via **getter** et **setter**.

Getter : lecture sécurisée des valeurs.

Setter : modification contrôlée avec validation.

Méthodes : définir des comportements sur l'objet (ici, afficher le solde).

```
// -----
// 3 ☐ Héritage : réutiliser le code
// -----
// Une classe peut hériter des attributs et méthodes d'une autre classe.
```

Placer chaque classe dans **son propre fichier** permet de **mieux organiser le code**.

- `lib/classes/animal.dart` → contient uniquement la classe **Animal** (la classe de base).
- `lib/classes/chien.dart` → contient uniquement la classe **Chien** (hérite de **Animal**).
- `lib/classes/chat.dart` → contient uniquement la classe **Chat** (hérite de **Animal**).

Avantages :

- Chaque fichier a **une seule responsabilité**, ce qui facilite la lecture et la maintenance.
- Les classes sont **réutilisables** dans différents projets ou modules.
- Réduit le risque de conflits ou d'erreurs liées aux importations.

2 ☐ Utilisation dans le fichier principal

Le fichier principal se trouve généralement dans `bin/` (ex : `bin/main.dart`).

Pour utiliser les classes que l'on a définies dans `lib/classes/`, on **importe** chaque fichier :

Résumé de la bonne pratique

- Une classe = un fichier → code clair et modulaire.
- Importer uniquement ce dont on a besoin dans le fichier principal.
- Facilite la **maintenance**, la **lecture** et la **réutilisation** du code.

```

/ Classe parent ou de base : Animal
class Animal {
    String nom; // Nom de l'animal

    // Constructeur de la classe Animal
    Animal(this.nom);

    // Méthode de base que tous les animaux peuvent utiliser
    void parler() {
        print("$nom fait un bruit."); // Message générique
    }
}

// Classe Chien qui hérite de Animal
class Chien extends Animal {
    // Constructeur : super() appelle le constructeur de la classe parente
    Chien(String nom) : super(nom);

    // Redéfinition (override) de la méthode parler() pour un comportement
    // spécifique
    @override
    void parler() {
        print("💡💡 $nom aboie : Ouaf Ouaf !");
    }
}

// Classe Chat qui hérite également de Animal
class Chat extends Animal {
    Chat(String nom) : super(nom);

    @override
    void parler() {
        print("💡💡 $nom miaule : Miaou !");
    }
}

// Fonction pour démontrer l'utilisation de l'héritage
void exempleHeritage() {
    // Création des objets Chien et Chat
    Chien chien = Chien("Rex");
    Chat chat = Chat("Mina");

    // Appel des méthodes spécifiques à chaque animal
    chien.parler(); // Affiche : Rex aboie

```

```

    chat.parler(); // Affiche : Mina miaule
}

```

Animal est la **classe de base** avec une méthode générique parler().
 Chien et Chat **héritent** de Animal et peuvent **redéfinir** (override) les méthodes pour un comportement spécifique.
 super() permet de **réutiliser le constructeur de la classe parente**.

```

// -----
// 4   □ Polymorphisme : un même nom, plusieurs comportements
// -----

```

Qu'est-ce que le polymorphisme ?

Le polymorphisme est un principe de la programmation orientée objet qui permet à des objets **de différentes classes liées par héritage** d'être traités de la même manière tout en exécutant leur propre comportement spécifique.

En d'autres termes :

- Même si on manipule tous les objets comme des Animal,
- Chaque objet exécute sa **version spécifique** de la méthode parler().

```

// On peut traiter différents objets (Chat, Chien...) comme un Animal,
// mais leur méthode parler() agit différemment.

```

```

void exemplePolymorphisme() {

```

```

    // ♦♦ Création d'une liste d'animaux polymorphes
    // La liste contient des objets de types différents : Chien, Chat et Animal
    List<Animal> animaux = [Chien("Max"), Chat("Luna"), Animal("Créature")];

```

```

    // ♦♦ Boucle sur chaque animal
    for (var animal in animaux) {
        // Polymorphisme : même si tous sont traités comme des Animal,
        // chaque objet exécute sa propre version spécifique de la méthode parler()
        animal.parler();
    }
}

```

```

// -----
// 5   □ Classe abstraite : modèle sans implémentation complète
// -----
// On ne peut pas créer d'objet à partir d'une classe abstraite,
// elle sert uniquement de plan pour d'autres classes.

```

```
// ❖❖❖ Définition d'une classe abstraite
// Une classe abstraite sert de modèle et ne peut pas être instanciée directement.
// Elle peut contenir des méthodes abstraites (sans implémentation)
// que les classes dérivées devront obligatoirement implémenter.
```

```
abstract class Vehicule {
    void demarrer(); // Méthode abstraite : la classe dérivée doit définir le comportement
    void arreter(); // Méthode abstraite : la classe dérivée doit définir le comportement
}
```

```
// ❖❖❖ Classe concrète qui hérite de la classe abstraite Vehicule
// La classe Voiture doit implémenter toutes les méthodes abstraites
class Voiture extends Vehicule {
```

```
    @override
    void demarrer() {
        print("❖❖❖ La voiture démarre !");
    }
```

```
    @override
    void arreter() {
        print("❖❖❖ La voiture s'arrête.");
    }
}
```

```
// ❖❖❖ Exemple d'utilisation de la classe abstraite via la classe concrète
void exempleClasseAbstraite() {
    Voiture v = Voiture(); // Instanciation de la classe concrète
    v.demarrer();           // Appel de la méthode implémentée
    v.arreter();            // Appel de la méthode implémentée
}
```

```
// -----
```

```
// 6   □ Interface : définir des contrats
```

```
// -----
```

```
// En Dart, une interface est simplement une classe que l'on implémente.
```

```
// ❖❖❖ Définition d'une interface en Dart
// En Dart, toute classe peut servir d'interface. Ici, la classe Paiement définit
// une méthode payer() que les classes qui l'implémentent devront
// obligatoirement fournir.
```

```
class Paiement {
    void payer(double montant) {}
}
```

```

}

// ❖❖❖ Classe qui implémente l'interface Paiement
// Cette classe doit définir le comportement concret de la méthode payer()
class PaiementMobile implements Paiement {
  @override
  void payer(double montant) {
    print("❖❖❖ Paiement mobile effectué de $montant FCFA via FlutterPay !");
  }
}

// ❖❖❖ Une autre implémentation de l'interface Paiement
class PaiementCarte implements Paiement {
  @override
  void payer(double montant) {
    print("❖❖❖ Paiement par carte bancaire de $montant FCFA réussi !");
  }
}

// ❖❖❖ Exemple d'utilisation de l'interface
// On peut manipuler plusieurs types de paiements via la même interface
// Paiement
void exempleInterface() {
  Paiement paiement1 = PaiementMobile(); // Paiement via mobile
  Paiement paiement2 = PaiementCarte(); // Paiement via carte bancaire

  // Chaque objet exécute sa propre version de la méthode payer()
  paiement1.payer(5000);
  paiement2.payer(12000);
}

// -----
// ❖❖❖ Fonction principale pour exécuter tous les exemples
// -----
void main() {
  print("❖❖❖ Bienvenue dans le monde de la Programmation Orientée Objet
(POO) en Dart !\n");

  exempleClasseEtObjet();
  print("\n-----\n");

  exempleEncapsulation();
  print("\n-----\n");

  exempleHeritage();
  print("\n-----\n");
}

```



```

exemplePolymorphisme();
print("\n-----\n");

exempleClasseAbstraite();
print("\n-----\n");

exempleInterface();

print("\n❖❖ Fin du cours sur la POO !");
}

```

```

// =====
// 6. Fonction asynchrone
// =====

```

❖❖ Qu'est-ce que la programmation asynchrone en Dart ?

La **programmation asynchrone** permet à ton programme **de continuer à s'exécuter sans attendre** qu'une opération longue soit terminée.

❖❖ En d'autres termes :

Le code n'attend pas qu'une tâche lente (comme un téléchargement ou un appel réseau) se termine pour passer à la suivante.

❖❖ Exemple de la vie réelle :

Imagine que tu commandes un café dans un restaurant :

- Tu passes commande
- Le serveur note ta commande et **continue de servir d'autres clients pendant que ton café est préparé**
- Quand ton café est prêt, **on t'appelle** pour venir le récupérer.

C'est **exactement ce que fait la programmation asynchrone** : elle **ne bloque pas** tout le programme pendant qu'une tâche est en attente.

Remarque : A ne pas confondre avec la Programmation synchrone, Le code s'exécute **ligne par ligne**, et chaque instruction **bloque la suivante** tant qu'elle n'est pas terminée.

Tu fais la queue à la poste : tu attends ton tour

En Dart :

Quand tu appelles une fonction asynchrone, elle te **renvoie immédiatement un "Future"**, c'est-à-dire une **promesse** qu'elle te donnera une valeur plus tard.

La programmation asynchrone en Dart est **essentielle** pour :

- effectuer des appels API sans bloquer ton interface Flutter,
- gérer des téléchargements, timers, animations,
- et créer des applications fluides et réactives.

On utilise Le fichier `async.dart` du dossier `lib` et le `main` du dossier `bin`

```
import 'dart:async'; // Pour les Futures, Timer, etc.
import 'package:http/http.dart' as http; // Pour les appels HTTP

// -----
// 🚧🚧 1   □ Point de départ : le main
// -----

void main() async {
  print("🚧🚧 Bienvenue dans le monde asynchrone de Dart !");

  // Exemple 1 : Code synchrone
  afficherMessageSynchrone();

  // 🚧🚧 Exemple 2 : Utilisation de Future.delayed pour simuler un délai
  // Cette fonction montre comment exécuter du code de manière asynchrone
  // sans bloquer le programme.
  await exempleFutureSimple();

  // 🚧🚧 Exemple 3 : Fonction asynchrone qui retourne une valeur
  // On utilise `await` pour récupérer le résultat d'une Future et l'afficher ensuite.
```

```

String valeur = await obtenirNomUtilisateur();
print("💎💎 Nom de l'utilisateur reçu : $valeur\n");

// 💎💎 Exemple 4 : Appel HTTP asynchrone pour récupérer des données
// depuis une API
// Cette fonction illustre l'utilisation de `http.get` et la gestion des erreurs avec
// try/catch.
await recupererDonneesDepuisAPI();

// 💎💎 Exemple 5 : Simulation d'un téléchargement de fichier
// On montre ici comment suivre la progression d'une opération asynchrone
// étape par étape.
await telechargerFichier();

print("💎💎 Fin du programme principal !");
}

// -----
// 💎💎 Exemple 1 : Code synchrone
// -----
void afficherMessageSynchrone() {
  print("💎💎 Exemple synchrone :");
  print("Étape 1 : Préparation...");
  print("Étape 2 : Traitement terminé !");
  print("  Aucun délai ici, tout s'exécute immédiatement.\n");
}

// -----
// 💎💎 Exemple 2 : Future.delayed pour simuler un délai
// -----

// 💎💎 Fonction asynchrone simple utilisant Future.delayed
// Cette fonction illustre comment Dart gère l'exécution asynchrone.
// Le mot-clé `async` permet d'utiliser `await` pour attendre des opérations
// futures.
Future<void> exempleFutureSimple() async {
  print("💎💎 Exemple asynchrone avec Future.delayed :");
  print("Attente de 2 secondes avant de continuer...");

  // 💎💎 Future.delayed crée un futur qui se termine après un certain délai.
  // Ici, il simule une opération longue, comme une requête réseau ou un
  // traitement lourd.
  await Future.delayed(Duration(seconds: 2));

  print("  2 secondes écoulées, exécution reprise.\n");
}

```

```
}
```

Future représente une valeur qui sera disponible plus tard.

async marque la fonction comme asynchrone.

await suspend l'exécution de la fonction jusqu'à ce que le futur soit complété.

Cela permet d'écrire du code non bloquant tout en gardant un style lisible, proche du code synchrone.

```
// -----  
// ❖❖❖ Exemple 3 : Une fonction asynchrone qui retourne une valeur  
// -----  
  
// ❖❖❖ Fonction asynchrone qui retourne un nom d'utilisateur  
// Cette fonction simule la récupération de données depuis une source externe  
// (API, base de données, serveur).  
// Le mot-clé `async` indique que la fonction renvoie un Future<String>.  
// Le mot-clé `await` est utilisé pour attendre la fin d'une opération asynchrone  
// sans bloquer l'exécution.
```

```
Future<String> obtenirNomUtilisateur() async {  
  print("❖❖❖ Récupération du nom de l'utilisateur en cours...");  
  
  // ❖❖❖ Simulation d'une latence de 3 secondes (comme une requête réseau)  
  await Future.delayed(Duration(seconds: 3));  
  
  // ❖❖❖ Valeur simulée reçue d'une source externe  
  String nom = "Chrislain Développeur";  
  
  // ❖❖❖ Retour de la valeur au futur appelant  
  return nom;  
}
```

La fonction retourne un Future<String> car la valeur n'est pas disponible immédiatement.

await permet de “mettre en pause” l'exécution de cette fonction jusqu'à ce que la donnée soit prête, sans bloquer le programme principal.

Cette approche est essentielle pour les applications Flutter/Dart qui font des appels réseau ou des opérations longues.

```
// -----  
// ❖❖❖ Exemple 4 : Appel HTTP pour récupérer des données d'une API  
// -----  
// ❖❖❖ Fonction asynchrone pour récupérer des données depuis une API
```

// Cette fonction montre comment effectuer un appel HTTP en Dart avec gestion des erreurs.
 // Le mot-clé `async` permet d'utiliser `await` pour attendre la réponse sans bloquer le programme principal.

```
import 'package:http/http.dart' as http;
Et importer la dépendance dans pubspec.yaml : le package http: ^1.1.0
```

```
Future<void> recupererDonneesDepuisAPI() async {
  print("💡💡 Exemple d'appel API en cours...");

  // 💡💡 URL de l'API à interroger
  final url = Uri.parse("https://jsonplaceholder.typicode.com/users/1");

  try {
    // 💡💡 Envoi de la requête GET et attente de la réponse
    final response = await http.get(url);

    // 💡💡 Vérification du code HTTP pour savoir si la requête a réussi
    if (response.statusCode == 200) {
      print("  Données API reçues avec succès :");
      print(response.body); // Affichage du contenu reçu
    } else {
      // 💡💡 Cas où le serveur retourne une erreur
      print("  Échec de la requête (code : ${response.statusCode})");
    }
  } catch (e) {
    // 💡💡 Gestion des exceptions (erreurs réseau, URL invalide, etc.)
    print("💡💡 Erreur lors de l'appel HTTP : $e");
  }

  print(""); // Ligne vide pour la lisibilité
}
```

```
// -----
// 💡💡 Exemple 5 : Simulation d'un téléchargement de fichier
// -----
```

```
// 💡💡 Fonction asynchrone pour simuler un téléchargement de fichier

// Cette fonction illustre l'utilisation de `Future.delayed` pour créer un délai
// et montrer la progression du téléchargement étape par étape.
Future<void> telechargerFichier() async {
  print("  Simulation de téléchargement d'un fichier...");
```

```
// ❖❖❖ Boucle pour simuler la progression du téléchargement
for (int i = 1; i <= 5; i++) {
    // Pause d'1 seconde pour simuler le temps de téléchargement
    await Future.delayed(Duration(seconds: 1));

    // ❖❖❖ Affichage de la progression en pourcentage
    print("Téléchargement : ${i * 20}%");
}

// Indication que le téléchargement est terminé
print(" Téléchargement terminé avec succès !\n");
}
```

Future.delayed permet de simuler un délai sans bloquer le programme principal. L'usage de await rend la fonction asynchrone non bloquante et permet de gérer les opérations dans un ordre séquentiel. Cette approche est similaire à ce que l'on ferait pour afficher la progression d'un vrai téléchargement dans une application Dart ou Flutter.

```
// -----
// ❖❖❖ Fonction principale pour exécuter tous les exemples asynchrones
// -----
```

```
// ❖❖❖ Exemple 2 : Utilisation de Future.delayed pour simuler un délai
void main async() {
```

```
// Cette fonction montre comment exécuter du code de manière asynchrone sans bloquer le programme.
await exempleFutureSimple();
```

```
// ❖❖❖ Exemple 3 : Fonction asynchrone qui retourne une valeur
// On utilise `await` pour récupérer le résultat d'une Future et l'afficher ensuite.
String valeur = await obtenirNomUtilisateur();
print("❖❖❖ Nom de l'utilisateur reçu : $valeur\n");
```

```
// ❖❖❖ Exemple 4 : Appel HTTP asynchrone pour récupérer des données depuis une API
```

```
// Cette fonction illustre l'utilisation de `http.get` et la gestion des erreurs avec
try/catch.
await recupererDonneesDepuisAPI();

//   Exemple 5 : Simulation d'un téléchargement de fichier
// On montre ici comment suivre la progression d'une opération asynchrone étape par
étape.
await telechargerFichier();

}
```

Explication à ajouter dans ton cours :

Chaque appel `await` permet d'attendre la fin d'une opération asynchrone avant de continuer.

Cela rend le code séquentiel et plus lisible tout en restant non bloquant pour le programme.

Les exemples montrent différentes utilisations : délai simulé, récupération de données, appel API et progression de téléchargement.

Conclusion:

Je te félicite d'avoir suivi ce cours de Dart jusqu'à la fin !

Le **meilleur moyen d'apprendre Dart**, c'est **d'écrire du code**. N'hésite pas à **réécouter cette vidéo tout en codant**, chapitre par chapitre, pour mieux assimiler chaque notion. Cela te permettra d'être pleinement à l'aise avec Dart et **de maîtriser Flutter** pour développer tes futures applications mobiles. N'oublie pas de **t'abonner à ma chaîne** pour rester à jour sur toutes les nouveautés en **Flutter et Dart** que je partage régulièrement !

